SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1704

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning,*
and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

Duration : 45 Minutes

QUESTIONS: 2 Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

*I **MITHRA DEVI S - 192524203** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
S. Mithra Devi

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q−table with zeros and state the hyperparameters (α, γ, ε). **(b)** Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

EXPERIMENT 1: DECISION TREE CLASSIFICATION

Aim

To implement and evaluate a **Decision Tree Classifier** using the Iris dataset and classify flowers based on their features.

Objective

- Load and preprocess the Iris dataset.
- Display the first five rows and data types.
- Split the dataset into training and testing sets.
- Build a Decision Tree using the **Gini Index**.
- Display the tree structure and identify the root node.
- Evaluate the model using accuracy, confusion matrix, and classification report.
- Identify misclassified instances.

Algorithm

1. Load the Iris dataset.
2. Convert the dataset into a Pandas DataFrame.
3. Check for missing values and data types.
4. Separate features and target variable.
5. Split the data into training and testing datasets.
6. Create a Decision Tree Classifier using the Gini criterion.
7. Train the model using the training data.
8. Predict the classes for the test data.
9. Calculate accuracy and confusion matrix.
10. Display precision, recall, and F1-score.
11. Identify incorrectly classified samples.
12. Display the learned decision-tree structure.

Python Code

```
# EXPERIMENT 1: DECISION TREE CLASSIFICATION

import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1. Load dataset
iris = load_iris()

# 2. Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target

print("FIRST 5 ROWS:")
print(df.head())

print("\nDATA TYPES:")
print(df.dtypes)

# 3. Check missing values
```

```
print("\nMISSING VALUES:")
print(df.isnull().sum())

# 4. Separate features and target
X = df.drop("target", axis=1)
y = df["target"]

# 5. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# 6. Create Decision Tree
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

# 7. Train model
model.fit(X_train, y_train)

# 8. Predict
y_pred = model.predict(X_test)

# 9. Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nACCURACY:")
print(accuracy)

# 10. Confusion Matrix
print("\nCONFUSION MATRIX:")
print(confusion_matrix(y_test, y_pred))

# 11. Classification report
print("\nCLASSIFICATION REPORT:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))

# 12. Print tree structure
```

```
print("\nDECISION TREE STRUCTURE:")
tree_rules = export_text(
    model,
    feature_names=list(X.columns)
)
print(tree_rules)

# 13. Root node
root_feature_index = model.tree_.feature[0]
root_feature = X.columns[root_feature_index]

print("\nROOT NODE:")
print(root_feature)

# 14. Find misclassified instances
print("\nMISCLASSIFIED INSTANCES:")

for index, actual, predicted in zip(
    X_test.index,
    y_test,
    y_pred
):
    if actual != predicted:
        print(
            "Index:", index,
            "Actual:", iris.target_names[actual],
            "Predicted:", iris.target_names[predicted]
        )
```

Output

First 5 Rows

```
FIRST 5 ROWS:

   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  target
0                5.1               3.5                1.4               0.2       0
1                4.9               3.0                1.4               0.2       0
2                4.7               3.2                1.3               0.2       0
3                4.6               3.1                1.5               0.2       0
4                5.0               3.6                1.4               0.2       0
```

Data Types

```
DATA TYPES:

sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target                 int64
dtype: object
```

Missing Values

MISSING VALUES:

sepal length (cm) 0
sepal width (cm) 0
petal length (cm) 0
petal width (cm) 0
target 0

Dataset Split

Training samples: 120
Testing samples: 30

Decision Tree Output

DECISION TREE STRUCTURE:

```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.65
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- sepal length (cm) <= 6.15
|   |   |   |   |--- sepal width (cm) <= 2.45
|   |   |   |   |   |--- class: 2
|   |   |   |   |--- sepal width (cm) > 2.45
|   |   |   |   |   |--- class: 1
|   |   |   |--- sepal length (cm) > 6.15
|   |   |   |   |--- class: 2
|   |--- petal width (cm) > 1.65
|       |--- petal length (cm) <= 4.85
|       |   |--- sepal width (cm) <= 3.00
|       |   |   |--- class: 2
|       |   |--- sepal width (cm) > 3.00
|       |   |   |--- class: 1
|       |--- petal length (cm) > 4.85
|           |--- class: 2
```

Root Node

ROOT NODE:
petal length (cm)

Justification

The **petal length** feature is selected as the root node because it provides the most effective initial separation of the three Iris classes according to the **Gini impurity criterion** used by the Decision Tree.

Model Evaluation

Accuracy

ACCURACY:
0.9333333333333333

Therefore,

Accuracy = 93.33%

Confusion Matrix

CONFUSION MATRIX:

```
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]
```

Classification Report

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

Misclassified Instances

MISCLASSIFIED INSTANCES:

Index: 134
Actual: virginica
Predicted: versicolor

Index: 77
Actual: versicolor
Predicted: virginica

Performance Analysis

The Decision Tree achieved **93.33% accuracy**, which indicates strong classification performance on the test data. The model correctly classified all **Setosa** samples. Two instances were misclassified between **Versicolor and Virginica**, which are more similar to each other than Setosa.

Result

The Decision Tree classifier was successfully implemented using the Iris dataset. The model achieved an accuracy of **93.33%** and correctly classified most test samples.

Conclusion

The experiment demonstrates that Decision Trees can effectively perform classification by selecting important features and recursively splitting the dataset. The Gini Index helped identify **petal length** as the root feature and produced a highly accurate model.

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

Aim

To implement **Q-Learning** for a 4 × 4 Grid World environment and learn an optimal policy through reinforcement learning.

Objective

- Define a 4 × 4 Grid World.
- Define states and actions.
- Assign rewards for goal, obstacle, and normal movement.
- Initialize the Q-table with zeros.
- Apply the Q-Learning algorithm for 100 episodes.
- Record Q-values at Episodes 1, 50 and 100.
- Extract the final learned policy.
- Plot cumulative reward per episode.
- Analyze convergence.

Environment Definition

The Grid World contains **16 states**.

```
+----+----+----+----+
| S0 | S1 | S2 | S3 |
+----+----+----+----+
| S4 | XX | S6 | S7 |
+----+----+----+----+
| S8 | S9 | XX | S11|
+----+----+----+----+
| S12| S13| S14| G  |
+----+----+----+----+
```

Where:

- **S0** = Starting state
- **G** = Goal state
- **XX** = Obstacle

Actions

0 → Up
1 → Down
2 → Left
3 → Right

Rewards

Situation	Reward
Reach Goal	+10
Normal movement	-1
Enter obstacle	-5

Hyperparameters

Parameter	Value
Learning Rate (α)	0.1
Discount Factor (γ)	0.9
Exploration Rate (ε)	0.1
Episodes	100

Q-Learning Formula

The Q-value is updated using:

$$Q(s,a) = Q(s,a) + \alpha[r + \gamma \max Q\,(s',a') - Q(s,a)]$$

Where:

- **α** = Learning rate
- **γ** = Discount factor
- **r** = Reward
- **s** = Current state
- **a** = Current action
- **s'** = Next state

Python Code

```
# EXPERIMENT 2: Q-LEARNING GRID WORLD

import numpy as np
import matplotlib.pyplot as plt

# --------------------------------
# Environment
# --------------------------------

GRID_SIZE = 4
NUM_STATES = 16
NUM_ACTIONS = 4

# Actions
UP = 0
DOWN = 1
LEFT = 2
RIGHT = 3

action_symbols = ["↑", "↓", "←", "→"]

start_state = 0
goal_state = 15

# Obstacles
obstacles = {5, 10}

# Hyperparameters
alpha = 0.1
gamma = 0.9
epsilon = 0.1
episodes = 100

# Initialize Q-table
Q = np.zeros((NUM_STATES, NUM_ACTIONS))
```

```
# Random generator
rng = np.random.default_rng(42)


# ------------------------------
# Environment Step Function
# ------------------------------

def step(state, action):

    row = state // GRID_SIZE
    col = state % GRID_SIZE

    if action == UP:
        new_row, new_col = row - 1, col

    elif action == DOWN:
        new_row, new_col = row + 1, col

    elif action == LEFT:
        new_row, new_col = row, col - 1

    else:
        new_row, new_col = row, col + 1

    # Boundary checking
    if (
        new_row < 0 or
        new_row >= GRID_SIZE or
        new_col < 0 or
        new_col >= GRID_SIZE
    ):
        return state, -1, False

    next_state = new_row * GRID_SIZE + new_col

    # Goal
    if next_state == goal_state:
        return next_state, 10, True

    # Obstacle
    if next_state in obstacles:
        return next_state, -5, False

    # Normal step
    return next_state, -1, False


# ------------------------------
```

```
# Training
# ------------------------------

rewards_per_episode = []
q_snapshots = {}

for episode in range(1, episodes + 1):

    state = start_state
    total_reward = 0

    for step_count in range(100):

        # Epsilon-greedy action selection
        if rng.random() < epsilon:
            action = int(rng.integers(NUM_ACTIONS))
        else:
            action = int(np.argmax(Q[state]))

        # Take action
        next_state, reward, done = step(state, action)

        # Q-learning update
        Q[state, action] = Q[state, action] + alpha * (
            reward +
            gamma * np.max(Q[next_state]) -
            Q[state, action]
        )

        total_reward += reward
        state = next_state

        if done:
            break

    rewards_per_episode.append(total_reward)

    # Store Q-table at specific episodes
    if episode in [1, 50, 100]:
        q_snapshots[episode] = Q.copy()


# ------------------------------
# Display Q-values
# ------------------------------

for episode in [1, 50, 100]:

    print("\nEpisode", episode)
```

```
    print("State 0:",
          np.round(q_snapshots[episode][0], 3))

    print("State 4:",
          np.round(q_snapshots[episode][4], 3))


# ------------------------------
# Final Policy
# ------------------------------

policy = np.zeros(NUM_STATES, dtype=int)

for state in range(NUM_STATES):

    if state == goal_state or state in obstacles:
        policy[state] = -1
    else:
        policy[state] = np.argmax(Q[state])

print("\nFINAL LEARNED POLICY:")

for row in range(GRID_SIZE):

    for col in range(GRID_SIZE):

        state = row * GRID_SIZE + col

        if state == goal_state:
            print(" G ", end=" ")

        elif state in obstacles:
            print(" X ", end=" ")

        else:
            print(
                action_symbols[policy[state]],
                end=" "
            )

    print()


# ------------------------------
# Plot Reward
# ------------------------------

plt.figure(figsize=(8, 5))
```

```
plt.plot(
    range(1, episodes + 1),
    rewards_per_episode
)

plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Cumulative Reward")
plt.grid(True)

plt.show()
```

Output

Initial Q-Table

Initially all Q-values are zero:

Q-table:

```
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 ...
 [0. 0. 0. 0.]]
```

Q-Table Evolution

The following values are obtained for two representative states, **State 0 and State 4**.

Episode 1

Episode 1

State 0: [-0.199 -0.280 -0.190 -0.190]

State 4: [-0.199 -0.280 -0.199 -0.500]

Episode 50

Episode 50

State 0: [-2.284 -2.181 -2.206 -2.147]

State 4: [-1.772 -1.685 -1.732 -1.756]

Episode 100

Episode 100

State 0: [-2.344 -2.215 -2.206 0.881]

State 4: [-1.772 -1.720 -1.732 -1.756]

The Q-values gradually change as the agent explores the environment and learns which actions lead toward the goal.

Final Learned Policy

FINAL LEARNED POLICY:

→ → → ↓

↓ X → ↓

→ ↓ X ↓

→ → → G

Meaning

→ = Move Right

↓ = Move Down

← = Move Left

↑ = Move Up

X = Obstacle

G = Goal

The learned policy generally guides the agent **toward the goal while avoiding obstacles**.

Cumulative Reward Output

The reward increases as the agent learns a better path.

Example final episodes:

Episode 91 → Reward = 5

Episode 92 → Reward = 5

Episode 93 → Reward = 5

Episode 94 → Reward = 5

Episode 95 → Reward = 4

Episode 96 → Reward = 5

Episode 97 → Reward = 5

Episode 98 → Reward = 5

Episode 99 → Reward = 5

Episode 100 → Reward = 5

The program generates the required **Cumulative Reward vs Episode graph** using Matplotlib.

Convergence Analysis

At the beginning, the agent has no knowledge about the environment, so the Q-table contains zeros and the agent explores different actions.

As the number of episodes increases:

- Q-values are updated repeatedly.
- The agent learns which actions produce better rewards.
- Unnecessary movements receive negative rewards.
- The path toward the goal becomes more favorable.
- The cumulative reward becomes more stable.

Therefore, the Q-learning agent demonstrates **learning and convergence toward a useful policy**.

Result

The Q-Learning algorithm was successfully implemented for a **4 × 4 Grid World** with a goal and obstacles. The Q-table was updated over **100 episodes**, and the agent learned a policy that guides it toward the goal while avoiding unfavorable actions.

Conclusion

This experiment demonstrates how **Reinforcement Learning** enables an agent to learn through trial and error. Q-Learning updates the Q-values based on rewards and gradually learns an optimal or near-optimal policy without being explicitly given the correct path.